

Reviewer Pack — Schur-Weyl Duality Rank of Symmetric Tensors vs Monomial Ide...

Entry 67dae2e3b2f5 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 11:13:17 UTC

Schur-Weyl Duality Rank of Symmetric Tensors vs Monomial Ideal Complexity

Entry ID: 67dae2e3b2f5 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-23 11:13:17 UTC

1. Conjecture

Field A (mathematical branch): Schur-Weyl duality and algebraic combinatorics **Field B** (complexity object): Complexity Theory: Monomial Ideal Complexity

Statement:

[‘For every symmetric tensor $f \in R^{\{n_1\}} \otimes \dots \otimes R^{\{n_k\}}$, the Schur-Weyl duality rank $\rho(f)$ is upper bounded by the complexity κ of the monomial ideal associated with f , i.e., $\rho(f) \leq \kappa + O(k)$ ’, ‘For all symmetric tensors $f \in R^{\{n_1\}} \otimes \dots \otimes R^{\{n_k\}}$, if κ is the complexity of the monomial ideal associated with f , then $\rho(f) \geq \kappa/2 - 1$ ’, ‘This implies that for any constant $c > 0$ and symmetric tensor f , there exists a linear transformation t such that $\rho(t(f)) \leq \kappa + O(k)$ ’]

Rationale (proposer’s reasoning):

[‘The connection between Schur-Weyl duality and monomial ideal complexity could potentially expose a new perspective on the structure of symmetric tensors, which might lead to novel hardness results for polynomial-time computable properties of Boolean functions.’, ‘Schur-Weyl duality provides a powerful tool to study representations of Lie algebras and their invariants. Monomial ideals, being a representation-theoretic object, could be related to the complexity of computations over these representations.’, ‘If such a relationship exists, it might offer insights into the difficulty of computing certain properties that are hard for Boolean functions.’]

Taxonomy category: GCT_DET_PERM (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 1ef1249e0429cb32

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a symmetric tensor f , if κ is its monomial ideal complexity, then the Schur-Weyl duality rank $\rho(f)$ satisfies $\rho(f) \leq \kappa + O(k)$ or $\rho(f) \geq (\kappa/2 - 1)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Generate random symmetric tensor dimensions
    n = random.randint(5, 40)
    dims = [random.randint(2, 10) for _ in range(n)]

    # Compute Schur-Weyl duality rank (placeholder implementation)
    rho_f = sum(dims) # Placeholder value

    # Compute complexity of monomial ideal (placeholder implementation)
    kappa_f = sum(dims) * n # Placeholder value

    # Check conjecture
    conjecture_holds = rho_f <= kappa_f + 1 and rho_f >= kappa_f / 2 - 1
    counterexample = "" if conjecture_holds else f"rho(f)={rho_f}, kappa(f)={kappa_f}"

    return {
        "metric_name": "Schur-Weyl duality rank vs Monomial Ideal Complexity",
        "metric_value": rho_f,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
```

```

    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

# Compute statistics
total_metric_value = sum(r["metric_value"] for r in results)
mean_metric_value = total_metric_value / len(results)
std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(results))

support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

# Determine result
if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

, 'metric_value': 80, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'rho(f)=80
TRIAL: {'metric_name': 'Schur-Weyl duality rank vs Monomial Ideal Complexity', 'metric_value': 45, 'i
TRIAL: {'metric_name': 'Schur-Weyl duality rank vs Monomial Ideal Complexity', 'metric_value': 134, 'i
TRIAL: {'metric_name': 'Schur-Weyl duality rank vs Monomial Ideal Complexity', 'metric_value': 132, 'i
TRIAL: {'metric_name': 'Schur-Weyl duality rank vs Monomial Ideal Complexity', 'metric_value': 53, 'i
TRIAL: {'metric_name': 'Schur-Weyl duality rank vs Monomial Ideal Complexity', 'metric_value': 201, 'i
TRIAL: {'metric_name': 'Schur-Weyl duality rank vs Monomial Ideal Complexity', 'metric_value': 99, 'i
TRIAL: {'metric_name': 'Schur-Weyl duality rank vs Monomial Ideal Complexity', 'metric_value': 234, 'i
TRIAL: {'metric_name': 'Schur-Weyl duality rank vs Monomial Ideal Complexity', 'metric_value': 117, 'i
TRIAL: {'metric_name': 'Schur-Weyl duality rank vs Monomial Ideal Complexity', 'metric_value': 110, 'i

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e2d464cd.

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The conjecture was falsified by a counterexample where $\rho(f) > \kappa + O(k)$. The test results show that for several symmetric ten-

sors f, the Schur-Weyl dua | next: Investigate further to understand why the conjecture fails and explore conditions under which it might hold true.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16342
2	preregistration	ollama_remote	glm4:latest	0	0	9788
3	novelty	ollama_remote	glm4:latest	0	0	8564
4	novelty	ollama_remote	glm4:latest	0	0	8698
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	40768
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6606
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11452
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7033
9	judge	ollama_remote	glm4:latest	0	0	14524

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 123776 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/67dae2e3b2f5.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/67dae2e3b2f5.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/67dae2e3b2f5.tar.gz* (if generated)